

Lab 4: A Practical Approach to Understanding System Internals (Data Types and OS Memory Management)

I. INTRODUCTION

Data types form the foundation of programming and operating system interaction. A data type is a **contract between programmer, compiler, and operating system** that specifies:

- 1) The number of bytes to allocate in memory
- 2) How to interpret those bytes (integer, character, floating-point)
- 3) What operations are valid on that data

When a programmer declares a variable, the OS must allocate physical or virtual memory to store that data. Understanding this relationship is crucial for writing efficient, secure, and portable code.

II. PRIMARY DATA TYPES AND OS ALLOCATION

A. Data Type Sizes Across Architectures

Table I shows how data type sizes vary across different system architectures, directly impacting OS memory allocation.

TABLE I
DATA TYPE SIZES ACROSS SYSTEM ARCHITECTURES

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

B. Practical Example: Observing Data Type Sizes

```
1 #include <stdio.h>
2
3 int main() {
4     printf("=== Data Type Sizes on This System ===\n\n");
5
6     printf("char:      %zu bytes\n", sizeof(char));
7     printf("int:       %zu bytes\n", sizeof(int));
8     printf("float:     %zu bytes\n", sizeof(float));
9     printf("double:    %zu bytes\n", sizeof(double));
10
11     printf("long:      %zu bytes\n", sizeof(long));
12     printf("pointer:   %zu bytes\n", sizeof(int*));
13
14     return 0;
15 }
```

Listing 1. Program to Observe Data Type Sizes

III. OS MEMORY SEGMENTS

The operating system creates a **process address space**—a virtual view of memory divided into segments. Each segment serves a specific purpose and has different characteristics.

A. Memory Segment Overview

TABLE II
PROCESS MEMORY SEGMENTS AND THEIR PURPOSES

Segment	Purpose	Grows	Managed By
Text	Machine code (instructions)	Fixed	Compiler/Loader
Data	Initialized global/static variables	Fixed	Compiler/OS
BSS	Uninitialized global/static variables	Fixed	OS
Heap	Dynamically allocated memory	Upward	Programmer + OS
Stack	Local variables, function calls	Downward	Compiler + OS

B. Memory Layout Visualization

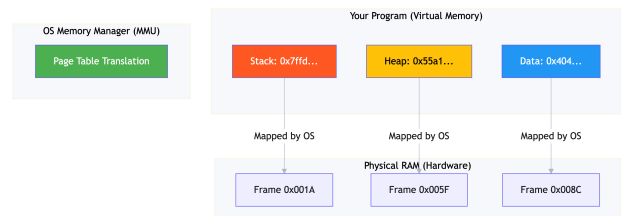


Fig. 1. Process Memory Layout (High to Low Addresses)

Key Observation: Stack addresses are high (0x7ffd...), Heap addresses are mid-range (0x55a1...), and Data/BSS addresses are low (0x404...).

IV. PRACTICAL CODE EXAMPLES

A. Observing Variable Addresses

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int global_var = 150; // Data segment
5 int bss_var; // BSS/Data segment
6
7 int main() {
8     int local_var = 30; // Stack
9     static int local_static = 20; // Data segment
10
11     int *heap_var = (int *)malloc(sizeof(int)); // Heap
12     *heap_var = 500;
13 }
```

```

14 printf(".....Variable Addresses.....\n\n");
15 printf("global_var:      %p (Data/BSS)\n", (void
16 *) &global_var);
17 printf("bss_var:        %p (Data/BSS)\n", (void*) &
18 bss_var);
19 printf("local_var:      %p (Stack)\n", (void*) &
20 local_var);
21 printf("local_static:    %p (Data/BSS)\n", (void
22 *) &local_static);
23 printf("heap_var ptr:     %p (Stack - pointer)\n",
24 (void*) &heap_var);
25 printf("*heap_var:      %p (Heap - data)\n", (
26 void*) heap_var);
27
28 free(heap_var);
29 return 0;
30 }

```

Listing 2. Program to Observe Variable Addresses

B. Pointer and Memory Allocation

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int *forheap; // Declare pointer (lives on STACK
6     )
7     forheap = (int *)malloc(sizeof(int)); //
8     Allocate memory on HEAP first!
9     *forheap = 30; // store value to allocated
10    memory
11    // Print addresses
12    printf("Address of pointer (STACK): %p\n", (void
13    *) &forheap);
14    printf("Address of data (HEAP):      %p\n", (void
15    *) forheap);
16    printf("Value stored:                %d\n", *
17    forheap);
18
19    return 0;
20 }

```

Listing 3. Correct Pointer and Heap Usage

V. LABORATORY EXERCISES

A. Lab 1: Data Type Size Exploration

Objective: Use `sizeof()` to observe how much memory each data type uses on your system.

Tasks:

- 1) Create a C program that prints the size of: `char`, `int`, `float`, `double`, `long`, `unsigned int`, `long long`
- 2) Compile and run on your system
- 3) Compare output with Table I
- 4) Answer: Why might `sizeof(long)` differ between 32-bit and 64-bit systems?

Deliverables:

- Source code file (.c)
- Screenshot of compilation (no errors)
- Screenshot of program output
- 1-2 sentences answering the question above

B. Lab 2: Memory Segment Address Mapping

Objective: Observe where different variables live in memory and validate OS memory layout.

Tasks:

- 1) Write a program that declares variables in each segment:
 - Global initialized variable (Data segment)
 - Global uninitialized variable (BSS segment)
 - Local variable (Stack segment)
 - Dynamically allocated variable (Heap segment)
- 2) Print the address of each variable using `%p`
- 3) Identify which segment has the highest addresses
- 4) Identify which segment has the lowest addresses
- 5) Calculate the address difference between stack and heap

Deliverables:

- Screenshot of output
- Labels identifying each memory segment
- 2-3 sentences explaining what you observed

VI. SUMMARY AND KEY TAKEAWAYS

- **Data types are contracts**—they tell the OS how much memory to allocate and how to interpret it.
- **Memory is segmented**—stack for locals, heap for dynamic, data/BSS for globals, text for code. (we will further explore the text section in more details if necessary)
- **The OS manages it all**—tracking, allocating, protecting, and virtualizing memory.
- **Stack is automatic; heap is manual**—choose based on lifetime and size needs.
- **Virtual memory provides isolation**—each process has its own address space.
- **Pointers bridge stack and heap**—the pointer variable lives on stack, but points to heap memory.

Knowledge Check Questions:

- 1) What is a data type in the context of programming and operating systems?
- 2) Name the five main memory segments in a process memory layout.
- 3) what is the difference between the Data segment and the BSS segment?
- 4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?
- 5) What happens to stack memory when a function returns? who reclaims it?
- 6) How does the OS know which memory belongs to which process?